MONITOR Z · L00.4

Clase 4 — Las piezas del monitor, y dónde estamos hoy

Cuaderno de estudio · versión del 2026-10-26

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L00.4 — Clase 4 · Las piezas del monitor, y dónde estamos hoy

Temario de la L00: 1. El demonio en Unix ✅ · 2. La misma idea en z/OS ✅ · 3. Por qué no alcanza con «preguntar cada tanto» ✅ · **4. Las piezas del monitor, y dónde estamos hoy**.

Objetivo de esta clase: tener el **mapa completo**, del job en DESA hasta la alerta en el portal, y saber qué pieza está probada, con qué JOBID, y qué falta. Reemplaza la sección 5 de la L00, que era del **2026-10-05** y quedó vieja.

> 📚 Fuentes: - `plataforma_cierres/vivo/PROTOCOLO.md`: el contrato y sus versiones 1 a 1.3. - `plataforma_cierres/vivo/puente.py` (cabecera) y `plataforma_cierres/app/vivo.py` (cabecera: umbrales y alertas). - `plataforma_cierres/vivo/validaciones/JOB34879_puente_13_desa.md` (2026-10-22): el cruce de los 12 eventos. - El *log* de este cerebro (P03, P04, P10) y la *Ruta de aprendizaje*.

1. Las piezas, de punta a punta

Analogía: una **cadena de radio de un guardia**. El guardia (ZTKMON) mira y habla por radio; el radio-operador (el puente) escucha, anota y le pasa los partes a la central (el portal); la central decide si algo es una alarma, y la pantalla grande (Balam) la muestra.

#	Pieza	Dónde vive	Qué hace	Testigo
1	🏢 **ZTKMON v0.9**	DESA, `PSJB.ZTK.LOAD`, job `ZTKMONR`	cada 5 s le pregunta a JES por SSI 80; manda `+EVT` numerados y los guarda en un anillo de 8.192	JOB34879, CC 0000

2	📜 **El protocolo 1.3**	`vivo/PROTOCOLO.md`	el **contrato**: texto EBCDIC con `X'15'`, `+HOLA`, `+EVT`, `SEQ=`, `DESDE`	4 versiones, cada una con su prueba
3	🕯️ **TCP, puerto 8010**	DESA ↔ PC	el cable; reservado para el job `ZTKMONR`	Sistemas, 2026-10-08
4	🖥️ **puente.py**	la PC	cliente TCP: latido, reconexión, `DESDE <último>`, y POST al portal	4 conexiones en JOB34879
5	🧠 **vivo.py** (portal)	la PC, :8767 para DESA	guarda, aprende el umbral de cada job (Kaplan–Meier) y abre alertas **COLGADO**, **FALLA** y **COLECTOR**	6 ZTKDOK terminados en `/api/vivo/estado`
6	📺 **Balam**	`/balam`, lee `/api/vivo/balam` cada 5 s	la consola del operador	IDEA-018
7	🗒️ **spool_a_portal.py**	la PC	el plan B: reproduce en el portal el SYSPRINT de un monitor sin puerto	modo spool (P04)

Una regla que atraviesa todo: el monitor **sólo mira**. El puente nunca le manda al host otra cosa que `SUB`, `ACT`, `JOB`, `HIST`, `DESDE`, `PING` y `ADIOS` (cabecera de `puente.py`, línea 18), y ninguno cambia nada en el host.



Lámina L00-21_las_piezas

2. El contrato: por qué las dos puntas se entienden

Versión	Fecha	Qué agregó	Probado con
1	2026-09-30	el contrato inicial	el simulador, 11/11
1.1	2026-10-08	reglas de `COMP`/`CC`, `TI=` opcional, la identidad de un job = jobid + `TI`	demo en DESA, JOB25639
1.2	2026-10-09	`+MET` y `+SIS` (CPU y memoria), opcionales	ZTKMON v0.6 (JOB26902/26905)
1.3	2026-10-22	`SEQ=`, `ARR=` en el `+HOLA`, el pedido `DESDE`, `+HUECO`	**JOB34879**

El truco que deja crecer sin romper: el `+HOLA` sigue diciendo `V=1`, porque cada versión sólo agrega campos **opcionales**, y el puente ignora las claves que no conoce. Un cambio incompatible pasaría a `V=2`. Por eso el puente viejo sigue andando contra un monitor nuevo, y por eso la A10 (`+PONG` con contadores) se puede agregar sin tocar el puente.

El contrato: cuatro versiones sin romper nada

L00 · Clase 4 · Las piezas y dónde estamos

Versión	Fecha	Qué agregó	Probado con
1	2026-09-30	el contrato inicial	el simulador, 11/11
1.1	2026-10-08	reglas de COMP/CC, `TI=` opcional, identidad = jobid + TI	DESA, JOB25639
1.2	2026-10-09	`+MET` y `+SIS`, opcionales	v0.6, JOB26902/26905
1.3	2026-10-22	`SEQ=`, `ARR=`, `DESDE`, `+HUECO`	**JOB34879**

🔑 El truco

El `+HOLA` sigue diciendo `V=1`: cada versión sólo agrega campos **opcionales**, y el puente ignora las claves que no conoce.

➡️ Para qué sirve

Un puente viejo anda contra un monitor nuevo. Por eso la **A10** (`+PONG` con contadores) se agrega sin tocar el puente.

⚠️ Cuándo no

Cambiar el formato de un campo, sacarlo o agregar uno **obligatorio** pasa a `V=2`.

Fuente: PROTOCOLO.md (sección Versiones)

Lámina L00-22_el_contrato

3. Un evento de punta a punta (JOB34879, 2026-10-22)

```
ZTKMON (DESA)                    puente.py (PC)                 porta
SSI 80 ve JOB34883 ejecutando
→ +EVT INI ZTKDOK JOB34883 … SEQ=1 ──TCP 8010──▶ lee hasta X'15' ──POST
  y lo guarda en el anillo                          lo pasa a JSON
… 30 s después
→ +EVT FIN … TI=11:40:28 COMP=NRM CC=0000 SEQ=3 ──────────────▶ ──────
```

Lo que pasó cuando se cortó el puente, tres veces a propósito:

Conexión	Pedido del puente	El monitor devolvió	Qué recuperó
1 · 11:37:43	`DESDE 0`	`N=00000`	nada todavía
2 · 11:42:16	`DESDE 4`	`N=00001`	el INI de JOB34887, que pasó durante el corte
3 · 11:50:08	`DESDE 8`	`N=00004`	INI y FIN de JOB34891 y JOB34892, que **corrieron enteros** durante el corte
4 · 11:53:55	`DESDE 12`	`N=00000`	nada que recuperar

Resultado: 12 de 12 eventos, 0 perdidos, 0 duplicados. 7 llegaron en vivo y **5 salieron del anillo**. Sin el anillo (antes de la v0.9), JOB34891 y JOB34892 no habrían existido para el portal.

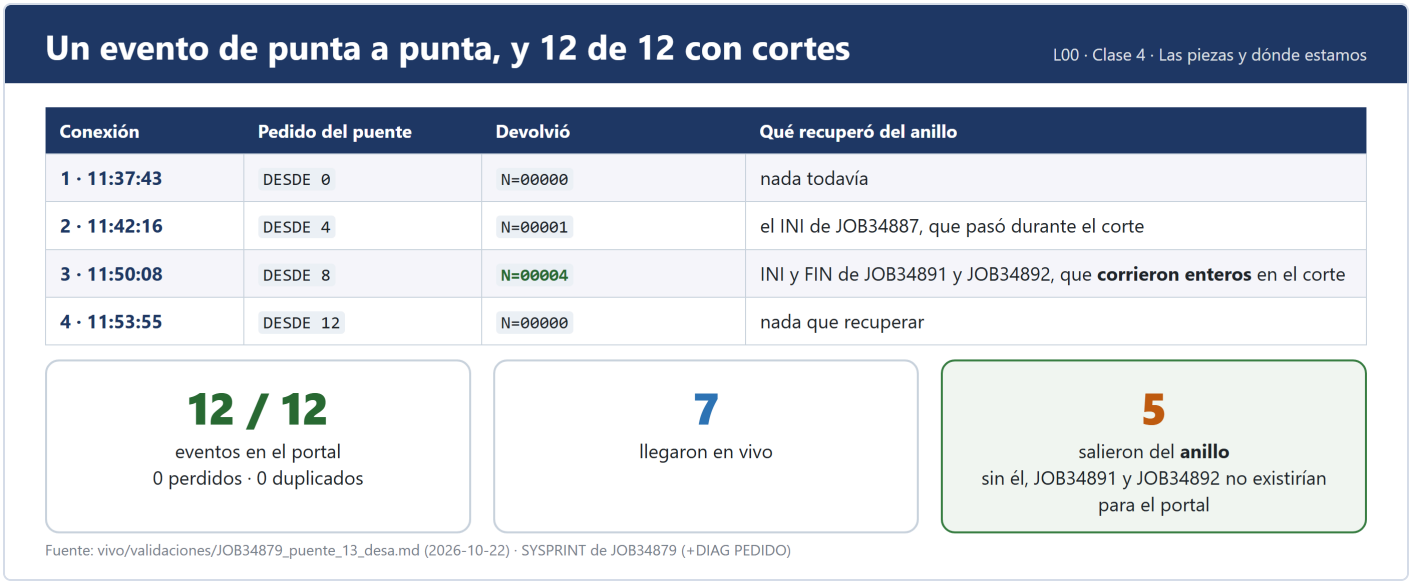


Lámina L00-23_doce_de_doce

4. Dónde estamos: la L00 del 05-10 contra hoy

Pieza	Lo que decía la L00 (2026-10-05)	Hoy (2026-10-28)	Testigo
Protocolo	✅ v1 escrito	✅ **1.3**, con 4 versiones probadas	`PROTOCOLO.md`
Puente y portal	✅ con el simulador, 11/11	✅ contra el **host real**; regresión 13/13 y anillo 7/7 con el simulador	JOB34879 · `prueba_e2e.py` · `prueba_anillo.py`
Programa del host	⚠️ ZTKMONS v0 escrito, **nunca ensamblado**	✅ **ZTKMON v0.9**: SSI 80, socket, filtro por dueño, `+MET`, `+PURGA`, `SEQ` y anillo	JOB34775 (RC 0) · JOB34879 (CC 0000)
Socket	⏳ no empezado	✅ en vivo por el 8010	P06b, JOB25949
STC	⏳	⏳ sigue siendo un job	L07, pide Sistemas

ENF 70	⏳	⏳	L08, pide APF
GQSCAN	⏳	⏳ (es la A5)	L09
Lugar para crecer	—	⚠️ quedan **1.920 bytes** en las 3 bases	JOB34775 · por eso la L15

Dónde estamos: la L00 del 05-10 contra hoy

L00 · Clase 4 · Las piezas y dónde estamos

Pieza	L00 (2026-10-05)	Hoy (2026-10-28)	Testigo
Protocolo	✅ v1 escrito	✅ **1.3, 4 versiones probadas**	PROTOCOLO.md
Puente y portal	✅ con el simulador	✅ **contra el host real**	JOB34879
Programa del host	⚠️ **nunca ensamblado**	✅ **ZTKMON v0.9**	JOB34775 · JOB34879
Socket	⏳	✅ **en vivo por el 8010**	P06b · JOB25949
STC · ENF 70 · GQSCAN	⏳	⏳ **siguen sin empezar**	L07 · L08 · L09
Lugar para crecer	—	⚠️ **1.920 bytes libres**	JOB34775 → L15

Las 5 preguntas abiertas de la L00, contestadas: 1 la fecha ✅ (P04) · 2 fase 14 ✅, y la 13 es «esperando» (P03) · 3 el JCL error en POSTEX ✅ (P04) · 4 `STATVRBO` sin autorización ✅, pero por `STATTRSA`: con filtros dio `REAS 116` · 5 ~6,5 ms (P04) y 6,71–6,95 ms hoy.

Fuente: L00, sección 5 · log de monitor-z (P03, P04, P10) · Ruta de aprendizaje · JOB34775 · JOB34879

Lámina L00-24_donde_estamos

5. Las cinco preguntas abiertas de la L00, contestadas

#	Pregunta (2026-10-05)	Respuesta	Testigo
1	¿`TIME DEC,…,DATETYPE=YYYYMMDD` deja la fecha en los bytes 8-11?	✅ sí	P04, JOB25373
2	¿Un job que ejecuta figura con fase 14 (`STAT_ONMAIN`)?	✅ sí; la **13** es «esperando», p. ej. por nombre repetido	P03, JOB25249 · P04
3	¿Un error de JCL aparece entre los terminados (`STAT_POSTEX`)?	✅ sí	P04, JOB25373
4	¿`STATVRBO` contesta a un programa sin autorización, con `STVBXTST` / `STVBXTET`?	✅ sí, pero **no** como se pidió al principio: con filtros dio `REAS 116`; anduvo pidiéndola por `STATTRSA` (v0.2). Hoy cada `+EVT FIN` trae `TI=`	P04 · JOB34879

5	¿Cuánta CPU gasta cada vuelta?	**~6,5 ms** con ZTKMONS; **6,71–6,95 ms** con ZTKMON v0.9	P04 · JOB34879

La pregunta 4 es el mejor ejemplo de por qué se prueba en DESA antes de dar algo por hecho: el manual decía que se podía, y se podía, pero no de la primera manera.

6. Lo que falta, y qué lección lo trae

Falta	Por qué importa	Lección	Depende de
Lugar en el programa	sin él no entra ninguna A más	**L15** (teoría ✅, código ⬜)	vos
Las mejoras A1–A12	ver más cosas del cierre	R02 y las que sigan	la L15
Arrancar solo y obedecer `P` / `F`	ser demonio de verdad, sin iniciador	**L07**	Sistemas: PROC y STARTED
Que JES avise	tardanza ~0 y ninguna vuelta en vano	**L08**	Sistemas: APF
El fin de cada paso con su RC	la bitácora en vivo (A7)	**L16** (propuesta: log stream)	Sistemas: SMF en log stream
Pasar a PROD	todo lo anterior, y AT-TLS (A11)	**L11**	gestión de cambios

Resumen

Idea	En una línea	Testigo
Las piezas	ZTKMON → protocolo → TCP 8010 → puente → vivo.py → Balam	JOB34879
Sólo mira	el puente sólo manda 7 pedidos, y ninguno cambia nada	`puente.py`, línea 18
El contrato	campos opcionales: `V=1` desde la v1 hasta la 1.3	`PROTOCOLO.md`
No perder eventos	12 de 12; 5 recuperados del anillo con `DESDE`	validación de JOB34879

Las 5 preguntas de la L00	contestadas; la 4, con un desvío (`STATTRSA`)	P03, P04, JOB34879
Lo que falta	lugar (L15), STC (L07), ENF 70 (L08), log stream (L16), PROD (L11)	la Ruta

Preguntas de comprobación

1. En la conexión 3 el puente pidió `DESDE 8` y recibió 4 eventos de dos jobs que corrieron **enteros** durante el corte.

¿Qué habría visto el portal de esos dos jobs con la v0.8, que no tenía anillo? ¿Y si el que se reinicia es el **monitor**, no el puente?

1. Elegí **una** de las piezas que faltan (sección 6) y explicá por qué depende de Sistemas o por qué no. ¿Cuál harías

primero, y por qué?

Mis respuestas

Relacionado: *L00 - Que es un demonio en z-OS y que vamos a construir* · *L00.3 - Clase 3 - Por que no alcanza con preguntar cada tanto* · *L14 - No perder eventos - numero de secuencia y anillo* · *R02 - Mapa de las mejoras A1-A12 contra ZTKMON v0.9* · *Ruta de aprendizaje*.

Notas